Lesson 69: Gated Recurrent Unit and Sequence Architectures

AI/ML Mathematics

Contents

1 Motivation and Overview **2**

2 References **3**

3 Gated Recurrent Unit **3**

3.1 GRU Architecture 3

3.2 GRU Gradient Flow 4

4 Bidirectional Recurrent Neural Networks **5**

5 Deep (Stacked) Recurrent Neural Networks **6**

6 Encoder-Decoder Framework **7**

7 Exercises **8**

8 Summary **9**

9 Further Reading **10**

1 Motivation and Overview

Lesson 67 established the vanilla RNN and exposed its fundamental limitation: vanishing gradients prevent learning long-range dependencies. Lesson 68 showed how the LSTM resolves this through an additive cell state update with learnable gates, at the cost of 4× more parameters. A natural question arises: *can we achieve comparable gradient flow with a simpler gating mechanism?*

The Gated Recurrent Unit (GRU), introduced by Cho et al. [2014], answers affirmatively. The GRU merges the cell state and hidden state into a single vector and uses only two gates (reset and update) instead of three. Empirical comparisons [Chung et al., 2014] show that the GRU achieves performance comparable to the LSTM on many tasks while requiring fewer parameters and less computation.

Beyond the choice of recurrent cell, the power of recurrent architectures is amplified by structural innovations: *bidirectional* processing captures both past and future context, *deep stacking* increases representational capacity, and the *encoder-decoder* framework enables variable-length sequence-to-sequence mappings. This lesson develops all four extensions and concludes with a capstone summary of the entire Recurrent Neural Networks subdomain.

Why GRU and Sequence Architectures Matter for RL/ML

- **Lightweight recurrent policies.** GRU-based policies offer a favorable trade-off between memory capacity and computational cost for on-policy RL algorithms like PPO that require many forward passes per training step.
- **Bidirectional context in offline RL.** When processing pre-collected trajectories (offline RL, imitation learning), bidirectional RNNs can leverage both past and future context to improve state estimation.
- **Deep recurrent architectures.** Stacked RNNs increase the capacity of recurrent value functions and policies, enabling richer representations in complex environments.
- **Sequence-to-sequence in RL.** The encoder-decoder framework underlies trajectory transformers, goal-conditioned policies, and communication protocols in multi-agent RL.

Prerequisites. LSTM cell architecture and gating mechanisms (Lesson 68, Definitions 4.1–4.7 and Theorem 7.1). Vanishing gradient problem and the cell state gradient highway (Lesson 68, Corollary 7.2 and Theorem 7.3). BPTT algorithm for vanilla RNNs (Lesson 67, Section 6).

Learning objectives. After completing this lesson, you will be able to:

- **Define** the GRU architecture and **derive** its forward equations, including the reset and update gates.
- **Compare** the GRU and LSTM in terms of parameter count, gradient flow properties, and computational efficiency.
- **Derive** the BPTT equations for the GRU, including the hidden state gradient recurrence.
- **Define** bidirectional RNNs and **explain** when bidirectional processing is appropriate.
- **Define** deep (stacked) RNNs and **compute** their parameter counts.
- **Formalize** the encoder-decoder framework and **explain** the context vector bottleneck.
- **Synthesize** the subdomain arc from vanilla RNNs through gated architectures to sequence-level designs.

2 References

- Cho et al. [2014] — Original GRU paper: the encoder-decoder architecture with GRU cells for statistical machine translation.
- Chung et al. [2014] — Empirical comparison of GRU and LSTM on sequence modeling tasks (polyphonic music, speech).
- Schuster and Paliwal [1997] — Bidirectional recurrent neural networks: architecture and training algorithm.
- Sutskever et al. [2014] — Sequence-to-sequence learning with neural networks: the encoder-decoder framework with LSTMs.
- Bahdanau et al. [2015] — Attention mechanism for encoder-decoder models, addressing the context vector bottleneck.
- Goodfellow et al. [2016, Chapter 10] — Comprehensive treatment of recurrent architectures and sequence modeling.

3 Gated Recurrent Unit

3.1 GRU Architecture

The GRU simplifies the LSTM by merging the cell state and hidden state into a single vector $\mathbf{h}_t$ and using two gates instead of three.

Definition 3.1 (Reset Gate)**.** The *reset gate* $\mathbf{r}_t \in (0,1)^m$ controls how much of the previous hidden state to "forget" when computing the candidate:

$$\boxed{\mathbf{r}_t = \sigma(\mathbf{W}_r\mathbf{x}_t + \mathbf{U}_r\mathbf{h}_{t-1} + \mathbf{b}_r)}, \tag{1}$$

where $\mathbf{W}_r \in \mathbb{R}^{m\times n}$, $\mathbf{U}_r \in \mathbb{R}^{m\times m}$, and $\mathbf{b}_r \in \mathbb{R}^m$.

Definition 3.2 (Update Gate)**.** The *update gate* $\mathbf{z}_t \in (0,1)^m$ controls the balance between the previous hidden state and the candidate:

$$\boxed{\mathbf{z}_t = \sigma(\mathbf{W}_z\mathbf{x}_t + \mathbf{U}_z\mathbf{h}_{t-1} + \mathbf{b}_z)}, \tag{2}$$

where $\mathbf{W}_z \in \mathbb{R}^{m\times n}$, $\mathbf{U}_z \in \mathbb{R}^{m\times m}$, and $\mathbf{b}_z \in \mathbb{R}^m$.

Definition 3.3 (GRU Candidate Hidden State)**.** The *candidate hidden state* $\tilde{\mathbf{h}}_t \in (-1,1)^m$ is computed using the reset gate to selectively incorporate the previous hidden state:

$$\boxed{\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h\mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)}, \tag{3}$$

where $\mathbf{W}_h \in \mathbb{R}^{m\times n}$, $\mathbf{U}_h \in \mathbb{R}^{m\times m}$, and $\mathbf{b}_h \in \mathbb{R}^m$.

Definition 3.4 (GRU Hidden State Update)**.** The hidden state is a convex combination of the previous hidden state and the candidate:

$$\boxed{\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t}. \tag{4}$$

GRU Forward Equations

At each time step $t = 1, \ldots, T$:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r), \quad \text{[Reset Gate]} \tag{5}$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z), \quad \text{[Update Gate]} \tag{6}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h), \quad \text{[Candidate]} \tag{7}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad \text{[Hidden State]} \tag{8}$$

Remark 3.5 (GRU vs LSTM: Structural Comparison). The GRU and LSTM share the principle of gated information flow but differ in key ways:

- The GRU has no separate cell state; $\mathbf{h}_t$ serves as both memory and output.
- The update gate $\mathbf{z}_t$ plays the role of both the LSTM's forget gate and input gate (coupled as $\mathbf{i} = 1 - \mathbf{f}$).
- The reset gate $\mathbf{r}_t$ has no direct LSTM analogue; it controls how much of $\mathbf{h}_{t-1}$ contributes to the candidate computation.
- The GRU hidden state update is a convex combination, so $\|\mathbf{h}_t\| \leq \max(\|\mathbf{h}_{t-1}\|, \|\tilde{\mathbf{h}}_t\|)$, providing implicit norm control.

Proposition 3.6 (GRU Parameter Count). *For a GRU with input dimension n and hidden dimension m, the total number of trainable parameters (excluding the output layer) is:*

$$P = 3(mn + m^2 + m) = 3m(n + m + 1). \tag{9}$$

Proof. Each of the three components (reset gate, update gate, candidate) has $mn + m^2 + m$ parameters. Total: $3(mn + m^2 + m) = 3m(n + m + 1)$. □

Example 3.7 (Parameter Comparison). For $n = 300$, $m = 512$ (word embedding to hidden state):

Architecture	Formula	Parameters
Vanilla RNN	$m(n + m + 1)$	416,256
GRU	$3m(n + m + 1)$	1,248,768
LSTM	$4m(n + m + 1)$	1,664,256

The GRU has 75% of the LSTM's parameters while often achieving comparable performance.

3.2 GRU Gradient Flow

Theorem 3.8 (GRU Hidden State Jacobian). *The Jacobian of $\mathbf{h}_t$ with respect to $\mathbf{h}_{t-1}$ in the GRU is:*

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{z}_t) + \mathbf{A}_t + \mathbf{B}_t + \mathbf{C}_t, \tag{10}$$

where:

$$\mathbf{A}_t = \operatorname{diag}(\mathbf{z}_t \odot (1 - \tilde{\mathbf{h}}_t^2))\, \mathbf{U}_h \operatorname{diag}(\mathbf{r}_t), \quad \textit{(candidate through reset)}$$

$$\mathbf{B}_t = \operatorname{diag}(\tilde{\mathbf{h}}_t - \mathbf{h}_{t-1}) \operatorname{diag}(\mathbf{z}_t \odot (1 - \mathbf{z}_t))\, \mathbf{U}_z, \quad \textit{(update gate)}$$

$$\mathbf{C}_t = \operatorname{diag}(\mathbf{z}_t \odot (1 - \tilde{\mathbf{h}}_t^2))\, \mathbf{U}_h \operatorname{diag}(\mathbf{h}_{t-1} \odot \mathbf{r}_t \odot (1 - \mathbf{r}_t))\, \mathbf{U}_r. \quad \textit{(reset gate)}$$

Proof. The hidden state update is $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$. Differentiating with respect to $\mathbf{h}_{t-1}$:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{z}_t) - \operatorname{diag}(\mathbf{h}_{t-1}) \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} + \operatorname{diag}(\tilde{\mathbf{h}}_t) \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} + \operatorname{diag}(\mathbf{z}_t) \frac{\partial \tilde{\mathbf{h}}_t}{\partial \mathbf{h}_{t-1}}.$$

For the update gate: $\frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\mathbf{z}_t \odot (1 - \mathbf{z}_t))\mathbf{U}_z$. For the candidate: $\frac{\partial \tilde{\mathbf{h}}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(1 - \tilde{\mathbf{h}}_t^2)\mathbf{U}_h(\text{diag}(\mathbf{r}_t) + \text{diag}(\mathbf{h}_{t-1})\frac{\partial \mathbf{r}_t}{\partial \mathbf{h}_{t-1}})$, where $\frac{\partial \mathbf{r}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\mathbf{r}_t \odot (1 - \mathbf{r}_t))\mathbf{U}_r$. Collecting terms yields $\mathbf{A}_t$, $\mathbf{B}_t$, and $\mathbf{C}_t$. □

Corollary 3.9 (GRU Gradient Preservation)**.** *When $\mathbf{z}_t \approx \mathbf{0}$ (update gate closed), the dominant term in the Jacobian is $\text{diag}(1 - \mathbf{z}_t) \approx \mathbf{I}$, and the hidden state approximately copies forward: $\mathbf{h}_t \approx \mathbf{h}_{t-1}$. This provides a gradient highway analogous to the LSTM's cell state pathway.*

GRU in Deep RL

In practice, GRUs and LSTMs perform comparably on most RL tasks. The GRU's lower parameter count makes it attractive for:

- On-policy methods (PPO, A3C) where many forward passes are needed per gradient update.
- Memory-constrained settings (mobile robotics, embedded systems).
- Multi-agent RL where each agent has its own recurrent network.

4 Bidirectional Recurrent Neural Networks

Definition 4.1 (Bidirectional RNN)**.** A *bidirectional RNN* processes a sequence in both forward and backward directions using two separate recurrent networks:

$$\overrightarrow{\mathbf{h}}_t = f_{\text{forward}}(\mathbf{x}_t, \overrightarrow{\mathbf{h}}_{t-1}), \tag{11}$$

$$\overleftarrow{\mathbf{h}}_t = f_{\text{backward}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}), \tag{12}$$

where f_{forward} and f_{backward} are recurrent cells (vanilla RNN, LSTM, or GRU) with independent parameters. The outputs are concatenated:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2m}, \tag{13}$$

where $[\cdot;\cdot]$ denotes vertical concatenation.

Proposition 4.2 (Bidirectional RNN Parameter Count)**.** *A bidirectional RNN with cell type C (having P_C parameters per direction) has $2P_C$ recurrent parameters plus the output layer parameters. The output layer maps from $2m$ dimensions instead of m.*

Proof. The forward and backward networks have independent parameters, so the total recurrent parameters are $2P_C$. The output layer $\mathbf{V} \in \mathbb{R}^{k \times 2m}$ maps the concatenated hidden state to the output. □

Example 4.3 (Bidirectional Context)**.** Consider named entity recognition in the sentence "John visited Paris last summer." At the word "Paris":

- The forward hidden state $\overrightarrow{\mathbf{h}}_3$ captures "John visited"—suggesting a destination.
- The backward hidden state $\overleftarrow{\mathbf{h}}_3$ captures "last summer"—suggesting a temporal context.

The concatenated representation $[\overrightarrow{\mathbf{h}}_3; \overleftarrow{\mathbf{h}}_3]$ captures both contexts, making it easier to classify "Paris" as a location.

Bidirectional RNNs Require Full Sequences

Bidirectional RNNs can only be applied when the *entire* sequence is available before prediction (the backward pass requires future inputs). They are **not suitable** for:

- Online prediction (each output must be produced before seeing future inputs).
- Causal sequence-to-sequence tasks (language modeling, real-time RL).

- Streaming applications where inputs arrive one at a time.

They **are suitable** for tasks like sequence classification, offline RL trajectory encoding, and post-hoc sequence labeling.

Remark 4.4 (Training Bidirectional RNNs). During training, the forward and backward networks are trained independently on their respective directions. The total gradient at each parameter is the sum of gradients from the forward and backward losses. Computationally, the forward pass requires two sequential scans of the sequence (one in each direction), doubling the forward pass time compared to a unidirectional RNN.

5 Deep (Stacked) Recurrent Neural Networks

Definition 5.1 (Stacked RNN)**.** An L-layer *stacked* (deep) RNN defines a hierarchy of recurrent layers. Let $\mathbf{h}_t^{(\ell)}$ denote the hidden state at time t and layer ℓ. The first layer processes the input:

$$\mathbf{h}_t^{(1)} = \mathrm{Cell}^{(1)}(\mathbf{x}_t, \mathbf{h}_{t-1}^{(1)}), \tag{14}$$

and subsequent layers use the hidden state of the layer below as input:

$$\mathbf{h}_t^{(\ell)} = \mathrm{Cell}^{(\ell)}(\mathbf{h}_t^{(\ell-1)}, \mathbf{h}_{t-1}^{(\ell)}), \quad \ell = 2, \ldots, L, \tag{15}$$

where $\mathrm{Cell}^{(\ell)}$ is a recurrent cell (vanilla RNN, LSTM, or GRU) with its own parameters. The output is typically taken from the top layer: $\mathbf{y}_t = g(\mathbf{V}\mathbf{h}_t^{(L)} + \mathbf{c})$.

Proposition 5.2 (Stacked RNN Parameter Count)**.** *For an L-layer stacked RNN where each layer has hidden dimension m and the cell type has parameter count $P_C(n_\ell, m)$ for input dimension n_ℓ:*

$$P = P_C(n, m) + (L-1)P_C(m, m) + k(m+1), \tag{16}$$

where the first term is the bottom layer (input dimension n), the middle term covers layers 2 through L (input dimension m), and the last term is the output layer.

Proof. Layer 1 receives input of dimension n, contributing $P_C(n, m)$ parameters. Layers $2, \ldots, L$ each receive hidden states of dimension m, contributing $(L-1)P_C(m, m)$ parameters. The output layer $\mathbf{V} \in \mathbb{R}^{k \times m}$ and $\mathbf{c} \in \mathbb{R}^k$ contribute $k(m+1)$. □

Example 5.3 (Stacked LSTM Parameter Count)**.** A 3-layer stacked LSTM with $n = 300$, $m = 512$, $k = 10000$ (vocabulary size):

$$\begin{aligned}
\text{Layer 1:} &\quad 4 \times 512 \times (300 + 512 + 1) = 1{,}664{,}256, \\
\text{Layers 2-3:} &\quad 2 \times 4 \times 512 \times (512 + 512 + 1) = 4{,}198{,}400, \\
\text{Output:} &\quad 10000 \times (512 + 1) = 5{,}130{,}000. \\
\text{Total:} &\quad 10{,}992{,}656 \approx 11\text{M}.
\end{aligned}$$

Remark 5.4 (Depth vs Width). Stacking layers adds *depth* in the spatial dimension (across layers at a fixed time step), complementing the temporal depth inherent in the recurrence. Each layer can learn increasingly abstract representations:

- Layer 1: low-level features (character patterns, phonemes).
- Layer 2: mid-level features (word meanings, phrases).
- Layer 3: high-level features (sentence semantics, discourse).

In practice, 2–4 layers suffice for most tasks; deeper stacks suffer from optimization difficulties similar to very deep feedforward networks.

Deep Recurrent RL

In deep RL, stacked LSTMs or GRUs are used when the agent faces complex partial observability. For example, a 2-layer LSTM in an Atari game with partial information: layer 1 processes visual features (from a CNN) into temporal patterns, while layer 2 integrates these patterns into a belief state for action selection.

6 Encoder-Decoder Framework

Definition 6.1 (Encoder-Decoder Architecture)**.** The *encoder-decoder* (sequence-to-sequence) architecture [Sutskever et al., 2014, Cho et al., 2014] maps a variable-length input sequence $\mathbf{x}_{1:T}$ to a variable-length output sequence $\mathbf{y}_{1:T'}$ (where $T' \neq T$ in general) through two components:

Encoder: processes the input sequence and produces a fixed-dimensional *context vector*:

$$\mathbf{h}_t^{\text{enc}} = f_{\text{enc}}(\mathbf{x}_t, \mathbf{h}_{t-1}^{\text{enc}}), \quad t = 1, \ldots, T, \tag{17}$$

$$\mathbf{q} = \phi(\mathbf{h}_T^{\text{enc}}), \tag{18}$$

where ϕ is typically the identity ($\mathbf{q} = \mathbf{h}_T^{\text{enc}}$).

Decoder: generates the output sequence conditioned on the context vector:

$$\mathbf{h}_t^{\text{dec}} = f_{\text{dec}}(\mathbf{y}_{t-1}, \mathbf{h}_{t-1}^{\text{dec}}), \quad \mathbf{h}_0^{\text{dec}} = \mathbf{q}, \tag{19}$$

$$\mathbf{y}_t = g(\mathbf{V}\mathbf{h}_t^{\text{dec}} + \mathbf{c}). \tag{20}$$

Remark 6.2 (Context Vector as Information Bottleneck). The context vector $\mathbf{q} \in \mathbb{R}^m$ must encode the *entire* input sequence into a fixed-dimensional vector. For long input sequences, this creates an information bottleneck: the encoder must compress arbitrarily long sequences into m dimensions, inevitably losing information. This limitation motivated the development of *attention mechanisms* [Bahdanau et al., 2015], which allow the decoder to access all encoder hidden states rather than just the final one.

Example 6.3 (Machine Translation)**.** In English-to-French translation of "The cat sat on the mat":

1. The encoder processes the English tokens one by one, building up $\mathbf{h}_1^{\text{enc}}, \ldots, \mathbf{h}_6^{\text{enc}}$.
2. The context vector $\mathbf{q} = \mathbf{h}_6^{\text{enc}}$ summarizes the entire English sentence.
3. The decoder generates French tokens autoregressively: "Le", "chat", "s'est", "assis", "sur", "le", "tapis", until a special end token is produced.

The output length (7 tokens) differs from the input length (6 tokens).

Proposition 6.4 (Encoder-Decoder Training)**.** *The encoder-decoder is trained end-to-end by maximizing the log-likelihood of the target sequence:*

$$\mathcal{L}(\boldsymbol{\theta}_{enc}, \boldsymbol{\theta}_{dec}) = -\sum_{t=1}^{T'} \log P(\mathbf{y}_t \mid \mathbf{y}_{1:t-1}, \mathbf{x}_{1:T}). \tag{21}$$

Gradients flow backward through the decoder, through the context vector, and into the encoder via BPTT.

Proof. The decoder output $\mathbf{y}_t$ depends on $\mathbf{h}_t^{\text{dec}}$, which depends on $\mathbf{h}_{t-1}^{\text{dec}}$ (via the decoder recurrence) and ultimately on $\mathbf{h}_0^{\text{dec}} = \mathbf{q} = \mathbf{h}_T^{\text{enc}}$. The encoder hidden state $\mathbf{h}_T^{\text{enc}}$ depends on the encoder parameters through the encoder recurrence. Thus the chain rule connects the decoder loss to the encoder parameters:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_{\text{enc}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_0^{\text{dec}}} \cdot \frac{\partial \mathbf{h}_T^{\text{enc}}}{\partial \boldsymbol{\theta}_{\text{enc}}}.$$

Both components are computed via BPTT in their respective networks. □

Encoder-Decoder in RL

The encoder-decoder framework appears in RL in several forms:

- **Goal-conditioned policies**: the encoder processes the goal description, and the decoder generates actions.
- **Trajectory transformers**: encode past trajectories and decode future actions or state predictions.
- **Communication in MARL**: agents encode messages using an encoder and decode received messages to inform actions (Phase 05, Lesson 64).

7 Exercises

Exercise 7.1 (GRU Forward Pass). (∗)
Consider a scalar GRU ($n = m = 1$) with all weight parameters equal to 1, all biases equal to 0, and $h_0 = 0$. Compute $r_1, z_1, \tilde{h}_1, h_1$ for input $x_1 = 1$. Then compute $r_2, z_2, \tilde{h}_2, h_2$ for input $x_2 = 0$.

Exercise 7.2 (GRU vs LSTM Gradient Analysis). (∗∗)
For a GRU with constant update gate $\mathbf{z}_t = \epsilon\mathbf{1}$ (small $\epsilon > 0$) and for an LSTM with constant forget gate $\mathbf{f}_t = (1 - \epsilon)\mathbf{1}$, compute the long-range gradient attenuation over k steps for each architecture. Show that both architectures preserve gradients when ϵ is small, but through different mechanisms.

Exercise 7.3 (Bidirectional RNN Output). (∗)
A bidirectional GRU with $m = 128$ per direction processes a sequence of length $T = 50$. What is the dimension of the output $\mathbf{h}_t$ at each time step? If the output layer maps to a vocabulary of $k = 10{,}000$ classes, how many parameters does the output layer have?

Exercise 7.4 (Encoder-Decoder Gradient Flow). (∗∗)
In an encoder-decoder with a vanilla RNN encoder (hidden dimension m) and LSTM decoder, derive the gradient of the decoder loss $\mathcal{L}_1^{\text{dec}}$ (at the first decoder step) with respect to the encoder hidden state $\mathbf{h}_1^{\text{enc}}$. Express your answer in terms of the encoder Jacobian products. Explain why the context vector bottleneck makes this gradient vanish for long input sequences when the encoder is a vanilla RNN, and how using an LSTM encoder helps.

Exercise 7.5 (Stacked GRU Design). (∗∗)
Design a 2-layer stacked GRU for a sentiment analysis task with input embedding dimension $n = 200$, hidden dimension $m = 256$ per layer, and $k = 5$ sentiment classes. (a) Compute the total parameter count. (b) If we use a bidirectional GRU for the first layer and a unidirectional GRU for the second layer, what changes in the parameter count and the input dimension of the second layer?

Exercise 7.6 (Attention Motivation). (∗ ∗ ∗)
Consider an encoder-decoder where the encoder produces hidden states $\mathbf{h}_1^{\text{enc}}, \ldots, \mathbf{h}_T^{\text{enc}}$ and the context is $\mathbf{q} = \mathbf{h}_T^{\text{enc}}$. Suppose instead we define a time-dependent context:

$$\mathbf{q}_t = \sum_{s=1}^{T} \alpha_{ts}\mathbf{h}_s^{\text{enc}}, \quad \alpha_{ts} = \frac{\exp(e_{ts})}{\sum_{s'=1}^{T}\exp(e_{ts'})}, \quad e_{ts} = \mathbf{v}^\top \tanh(\mathbf{W}_a\mathbf{h}_s^{\text{enc}} + \mathbf{U}_a\mathbf{h}_{t-1}^{\text{dec}}).$$

This is the attention mechanism of Bahdanau et al. [2015]. (a) Show that when $\alpha_{ts} = \delta_{sT}$ (attention focuses entirely on the last encoder state), this reduces to the standard encoder-decoder. (b) Explain how attention alleviates the information bottleneck. (c) Derive $\frac{\partial \mathbf{q}_t}{\partial \mathbf{h}_s^{\text{enc}}}$ and

argue that attention provides a "shortcut" for gradient flow from the decoder to early encoder states.

Hint: For part (c), note that the gradient of $\mathbf{q}_t$ with respect to $\mathbf{h}_s^{\text{enc}}$ includes a direct term $\alpha_{ts}\mathbf{I}$ that does not pass through the encoder recurrence.

8 Summary

- The **GRU** simplifies the LSTM by merging cell and hidden states and using two gates (reset and update) instead of three, achieving 75% of the LSTM's parameter count with comparable performance.
- The GRU hidden state update $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$ is a convex combination that provides gradient preservation when $\mathbf{z}_t \approx \mathbf{0}$.
- **Bidirectional RNNs** process sequences in both directions, capturing past and future context, but require the entire sequence to be available (not suitable for online or causal settings).
- **Stacked (deep) RNNs** add spatial depth by layering recurrent cells, enabling hierarchical feature extraction. In practice, 2–4 layers suffice.
- The **encoder-decoder** framework maps variable-length input sequences to variable-length output sequences through a context vector, enabling tasks like machine translation.
- The context vector creates an **information bottleneck** for long sequences, motivating attention mechanisms that allow the decoder to access all encoder hidden states.

Subdomain 3.2 Summary: Recurrent Neural Networks

The following table summarizes the arc of Lessons 67–69.

Lesson	Topic	Key Results	Significance
67	Vanilla RNN and BPTT	RNN forward equations; BPTT algorithm; vanishing gradient bound $\|\partial\mathcal{L}_t/\partial\mathbf{h}_\tau\| \leq C\gamma^{t-\tau}$; exploding gradient bound	Establishes the fundamental architecture and its core limitation
68	LSTM and Gating	LSTM forward equations (6 equations); cell state Jacobian $\partial\mathbf{c}_t/\partial\mathbf{c}_{t-1} = \text{diag}(\mathbf{f}_t)$; gradient highway when $\mathbf{f}_t \approx \mathbf{1}$	Solves the vanishing gradient problem via additive cell state update
69	GRU and Sequence Architectures	GRU equations (4 equations); GRU Jacobian; bidirectional concatenation; stacked layers; encoder-decoder framework	Simplified gating + structural innovations for practical sequence processing

Unifying theme: the evolution from multiplicative (vanilla RNN) to additive (LSTM) to simplified additive (GRU) gradient flow, complemented by structural innovations (bidirectional, stacked, encoder-decoder) that expand the representational and functional capabilities of recurrent architectures.

9 Further Reading

- The attention mechanism of Bahdanau et al. [2015] resolves the encoder-decoder bottleneck and leads directly to the Transformer architecture, which has largely replaced RNNs in NLP. A mathematical treatment of attention and Transformers would be a natural extension of this subdomain.
- Chung et al. [2014] provides empirical comparisons of GRU and LSTM on music and speech modeling, finding no clear winner across all tasks.
- Greff et al. [2017] is the most comprehensive ablation study of LSTM variants, testing the importance of each gate and providing practical recommendations.
- Phase 04 (Deep RL) builds on recurrent architectures: recurrent DQN (DRQN) uses LSTMs in value-based methods, while recurrent PPO uses LSTMs/GRUs for policies in POMDPs.
- Graves [2012] covers sequence transduction with RNNs, including connectionist temporal classification (CTC) for speech recognition and handwriting synthesis.
- The historical development of gated RNNs traces from Hochreiter [1991]'s identification of the vanishing gradient problem, through Hochreiter and Schmidhuber [1997]'s LSTM solution, Gers et al. [2000]'s addition of the forget gate, to Cho et al. [2014]'s GRU simplification.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.

Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. In *NIPS Workshop on Deep Learning*, 2014.

Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

Alex Graves. *Supervised Sequence Labelling with Recurrent Neural Networks*. Springer, 2012.

Klaus Greff, Rupesh Kumar Srivastava, Jan Koutník, Bas R. Steunebrink, and Jürgen Schmidhuber. LSTM: A search space odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10):2222–2232, 2017.

Sepp Hochreiter. Untersuchungen zu dynamischen neuronalen netzen. Diploma thesis, Technische Universität München, 1991.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.

Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, volume 27, pages 3104–3112, 2014.